Tetta AI

Phase 1: Mid-Semester Project (The MVP) - Code Use

STUDENT VIEW

8ce00921-f923-41fe-8657-6b34a3f5913b

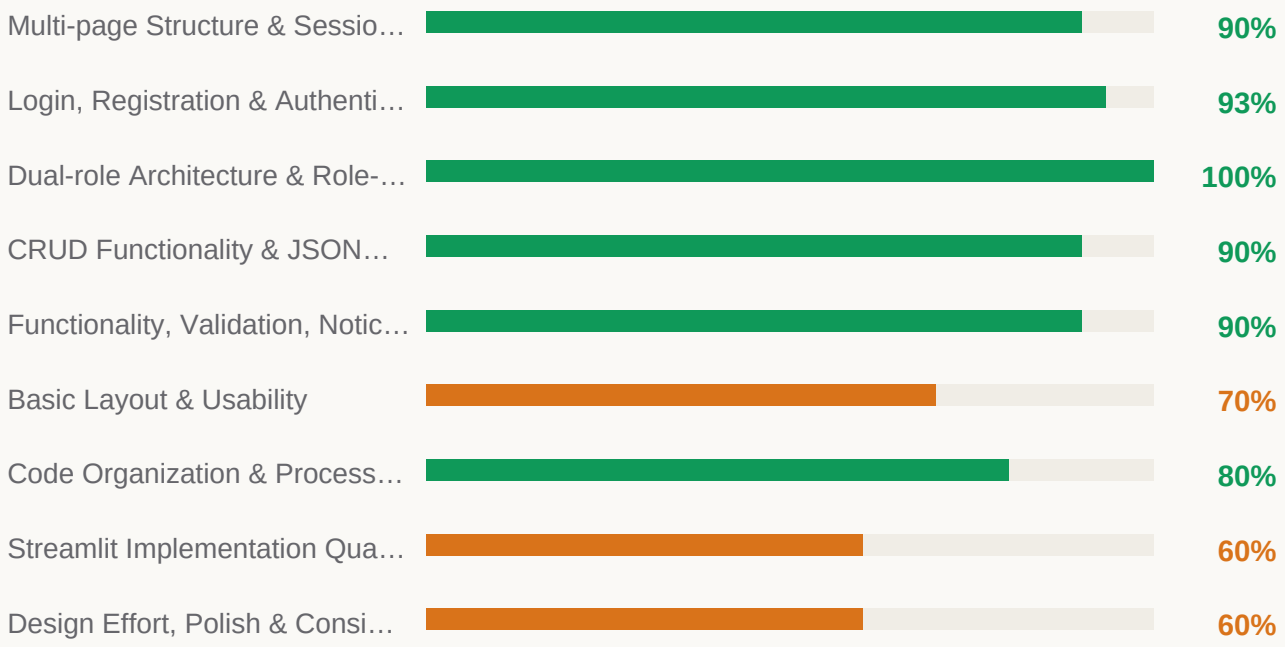
Attempt 3 • Apr 5, 2026, 10:28 PM • Apr 5, 2026, 10:32 PM

85/100

Solid, functional Streamlit MVP that implements multi-page navigation, two user roles, persistent JSON storage, and end-to-end flows. Most core features work reliably; a few UX/robustness and Streamlit best-practice issues remain (minor session/rerun edge cases, some UI state keys, and design polish).

Rubric Overview

Score distribution across all criteria



Rubric Breakdown

Multi-page Structure & Session-State Navigation

9 / 10

Navigation is clear and driven by session_state with distinct views for guests and logged-in users; logout and rerun logic generally work. Minor edge cases exist where some form actions call st.rerun() while others rely on implicit state updates, occasionally leaving transient stale UI.

Login, Registration & Authentication Flow

14 / 15

Registration and login are implemented with password hashing, duplicate username checks, and validation for empty/mismatched inputs. Consider adding stronger feedback for password rules and protecting against timing/file errors, but core authentication works correctly.

Dual-role Architecture & Role-based Feature Structure

15 / 15

Two distinct roles (architect and collaborator) are present with clearly different feature sets and role-specific navigation; routing after login is correct. The app structure reflects role differences well and permissions are enforced in the UI.

CRUD Functionality & JSON Persistence

18 / 20

Create, read, update, and delete flows for events, needs, claims, and passes persist to JSON files and related cascading deletions (passes when events/needs removed) are handled. Ensure robust file I/O (atomic writes/locking) and consider validation to guarantee saved data always matches UI state in concurrent scenarios.

Functionality, Validation, Notices & User Flow

9 / 10

Main workflows complete with appropriate success/error notices, validation on forms, and helpful messages when lists are empty. A few input/race-condition edge cases (e.g., simultaneous claims) and occasional inconsistent rerun behavior could make the app feel fragile under stress.

Basic Layout & Usability

7 / 10

Layout uses sidebar, columns, tabs, and expanders to organize functionality which makes the app usable and reasonably readable. Visual polish and consistent hierarchy can be improved (spacing, clearer summaries on hubs, and avoiding markdown-heavy assistant replies) for a more dashboard-like feel.

Code Organization & Process Separation

4 / 5

Helpers for load/save, hashing, id generation, and lookups separate concerns and the main flows are readable rather than mashed together. Some repeated patterns in forms and per-item UI could be refactored into functions to reduce duplication and improve maintainability.

Streamlit Implementation Quality & Best Practices

3 / 5

Most Streamlit idioms are used correctly (forms, keys, session_state), and rerun is used where needed in some spots. However, some widget keys are reused across dynamic lists and a few UI updates rely on implicit behavior; standardizing keys and ensuring no duplicate widget IDs will make behavior more stable.

Design Effort, Polish & Consistency (Design Excellence)

6 / 10

Good baseline design effort with tabs, expanders, and helpful captions; the app is functional and approachable. To reach higher polish, unify typography/spacing, refine the sidebar hierarchy, and reduce text-heavy assistant outputs in favor of concise itemized UI.

Strengths

- Complete role-based flows: registration/login, architect event management, collaborator claiming, and pass management all work end-to-end.
- Persistent JSON storage for users, events, and passes with save/load helpers; CRUD operations write back to the files.
- Good separation of concerns in many helper functions (load/save, hashing, id creation, event lookup) and clear session-state initialization.
- Thoughtful UX touches such as balloons on successful claim, success/error notices, and forms to avoid inline partial updates.

Needs Attention

- Avoid relying on implicit st.rerun() behavior: some button/form combinations trigger reruns while others leave stale UI or duplicate keys (selectbox/number_input keys per event).
- Tighten Streamlit widget key usage to ensure unique keys and avoid potential collisions when many events/needs exist.
- Polish layout consistency: use clearer grouping/spacing (sidebars, columns) and reduce markdown/raw JSON exposure in the UI for a cleaner dashboard feel.
- Add defensive checks around file I/O (concurrent writes) and better handling when related objects are missing to avoid runtime errors.